

Inheritance

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

1. Method Overriding.....	1
2. Abstract Classes and Abstract Methods	2
3. Dynamic Method Dispatch.....	4
4. Final Keyword in Inheritance.....	5
5. Summary of Key Concepts.....	5
6. References.....	6

6.2.1 Method Overriding

✓ Definition:

Method overriding allows a subclass to provide a specific implementation of a method already defined in its superclass.

✓ Rules for Method Overriding:

- The method in the child class must have the **same name, return type, and parameters**.
- The **access modifier** must be the same or **more visible** (not more restrictive).
- The method **cannot be overridden if it's marked final**, static, or private.
- **Constructors** cannot be overridden.

✓ Example:

```
class Animal {  
    void sound() {  
        System.out.println("Animal makes a sound");  
    }  
}  
  
class Dog extends Animal {  
    @Override  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}
```

◆ Why Use Method Overriding?

- To provide **specific implementation** of a method already defined in the superclass.
- To achieve **dynamic method dispatch** (polymorphism at runtime).
- To allow Java to determine the method to invoke **based on object type, not reference type**.

6.2.2 Abstract Classes and Abstract Methods

Abstract Classes and Abstract Methods in Java are used when you want to define a base class that cannot be instantiated directly but can be subclassed by other classes to provide specific implementations. They are key components for achieving **abstraction**, which is one of the core principles of Object-Oriented Programming.

◆ Abstract Class

An **abstract class** is a class that cannot be instantiated on its own. It may contain abstract methods (methods without a body) or concrete methods (with implementations). Abstract classes are meant to be extended by other classes that provide specific implementations for the abstract methods.

- **Declaration:** Use the abstract keyword.
- **Purpose:** To provide a common base for subclasses to share some common code and force them to implement certain methods.
- **Abstract Methods:** An abstract class can contain **abstract methods** (methods without implementation), which must be implemented by its subclasses.
- **Concrete Methods:** An abstract class can also contain **concrete methods** (methods with implementation) that can be inherited directly.

◆ Abstract Method

An **abstract method** is a method that is declared without an implementation (without a body). The method signature is specified, but the subclass is expected to provide the implementation.

- **Declaration:** It must be declared in an abstract class or an interface.
- **Implementation:** A subclass that inherits an abstract class must provide the implementation for the abstract methods.

◆ Syntax of Abstract Class and Abstract Method

```
// Abstract class with an abstract method
abstract class Animal {
    // Concrete method
    void breathe() {
        System.out.println("Breathing...");
    }

    // Abstract method (no body)
    abstract void sound();
}

// Subclass that provides implementation for the abstract method
class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}
```

```
public class Main {
    public static void main(String[] args) {
        Animal a = new Dog(); // Cannot instantiate Animal directly
        a.sound();           // Output: Dog barks
        a.breathe();         // Output: Breathing...
    }
}
```

◆ Key Points of Abstract Classes and Methods

Feature	Abstract Class	Abstract Method
Keyword	abstract	abstract
Instantiation	Cannot be instantiated	N/A
Methods	Can have both abstract and concrete methods	Must not have a body, only declaration
Constructor	Can have constructors	Cannot have a constructor
Subclasses	Must be inherited by concrete classes	Must be implemented in subclass
Access Modifiers	Can have any access modifier	Can be public, protected, or private
Purpose	To provide common functionality	To define method signatures to be implemented

◆ When to Use Abstract Classes and Methods:

- When you want to define **shared functionality** in the base class but **force subclasses** to provide specific implementations for some methods.
- When you want to define a **common interface** for subclasses while still providing default implementations for some methods.
- To **avoid object instantiation** for the base class, ensuring only subclasses can be instantiated.

6.2.3 Dynamic Method Dispatch

Dynamic Method Dispatch in Java refers to the mechanism by which a call to an overridden method is resolved at **runtime**, rather than at compile time. This is a core feature of **runtime polymorphism** and is used to determine which method implementation to invoke based on the object type, not the reference type.

◆ How Dynamic Method Dispatch Works:

- When a method is **overridden** in a subclass, Java uses the **runtime object type** to determine which method to call, not the type of reference variable.
- This allows Java to dynamically determine the method that should be executed, depending on the actual object that the reference variable is pointing to, even if the reference is of a superclass type.

◆ Example:

Let's look at an example that demonstrates dynamic method dispatch:

```
class Animal {
    // Method in the parent class
    void sound() {
        System.out.println("Animal makes a sound");
    }
}

class Dog extends Animal {
    // Method overridden in the subclass
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

class Cat extends Animal {
    // Method overridden in the subclass
    @Override
    void sound() {
        System.out.println("Cat meows");
    }
}

public class Main {
    public static void main(String[] args) {
        Animal animal1 = new Dog(); // Reference of type Animal, object of type Dog
        Animal animal2 = new Cat(); // Reference of type Animal, object of type Cat
    }
}
```

```
        animal1.sound(); // Output: Dog barks
        animal2.sound(); // Output: Cat meows
    }
}
```

◆ Key Features of Dynamic Method Dispatch:

1. **Runtime Polymorphism:** It is a way to achieve runtime polymorphism, where the method call is resolved dynamically at runtime based on the object type.
2. **Method Overriding:** It works only when a method in a superclass is **overridden** by a subclass.
3. **Flexibility:** The type of object is determined at runtime, allowing more flexible and extensible code.

6.2.4 Final Keyword in Inheritance

✓ Final Method:

- A method declared with final **cannot be overridden** in subclasses.

```
class Parent {
    final void display() {
        System.out.println("Final method");
    }
}
```

✓ Final Class:

- A class declared as final **cannot be extended**.

```
final class Vehicle {
    void run() {
        System.out.println("Running...");
    }
}
```

➤ Summary of Key Concepts:

- Method overriding allows a subclass to modify the behavior of a superclass method.
- Abstract classes provide a template for future subclasses with abstract and concrete methods.
- Dynamic method dispatch enables polymorphic behavior at runtime.
- The final keyword is used to restrict method overriding and inheritance.

➤ References:

- Oracle Java Documentation – Method Overriding
🔗 <https://docs.oracle.com/javase/tutorial/java/IandI/override.html>
- GeeksforGeeks – Method Overriding in Java
🔗 <https://www.geeksforgeeks.org/method-overriding-in-java/>
- Oracle Java Documentation – Abstract Classes and Methods
🔗 <https://docs.oracle.com/javase/tutorial/java/IandI/abstract.html>
- GeeksforGeeks – Abstract Classes and Methods in Java
🔗 <https://www.geeksforgeeks.org/abstract-classes-and-methods-in-java/>
- Oracle Java Documentation – Final Keyword
🔗 <https://docs.oracle.com/javase/tutorial/java/javaOO/final.html>
- GeeksforGeeks – Final Keyword in Java
🔗 <https://www.geeksforgeeks.org/final-keyword-in-java/>
- Oracle Java Documentation – Polymorphism
🔗 <https://docs.oracle.com/javase/tutorial/java/IandI/polymorphism.html>
- GeeksforGeeks – Dynamic Method Dispatch in Java
🔗 <https://www.geeksforgeeks.org/dynamic-method-dispatch-in-java/>

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | GRADE



    
<https://paruluniversity.ac.in/>